

Basic NLGMM example

James Foster

2026-06-02

Contents

Basic Nonlinear GLMM example	1
Simulate data	1
Save and load data	4
Plot the data	4
Install and set up Bayesian modelling packages	5
Define the nonlinear model	6
Set custom priors	6
Fit the model	9
Inspect the posterior fit	14
Model comparison and predictions	16

```
set.seed(17070523)
knitr::opts_chunk$set(echo = TRUE)
```

Basic Nonlinear GLMM example

This example simulates nonlinear psychometric data and then fits models including a Bayesian nonlinear model using `brms`.

Simulate data

We construct a dataset with stimulus intensity, stimulus type, and a binomial response.

```
n_animals <- 10
n_levels <- 7
n_types <- 2
replicates <- 20

width_alpha <- 0.8
width_coef <- 2 * log(2 / (1 - width_alpha) - 1)
inflex <- 15
width <- -10
```

```

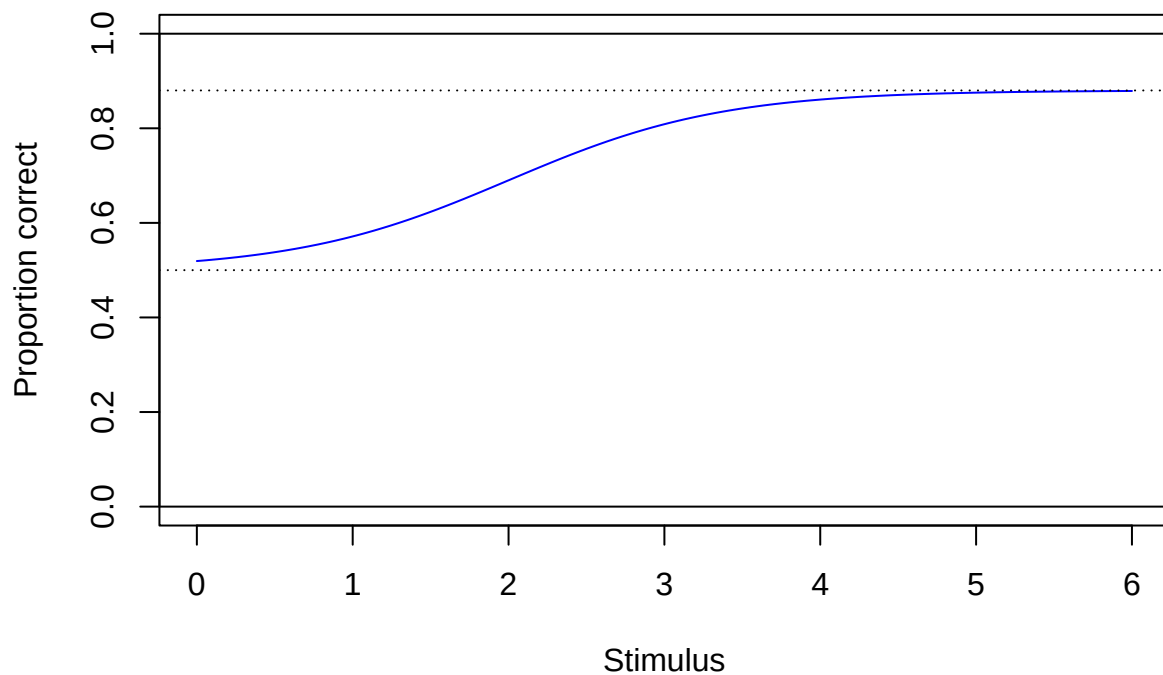
lapse <- 0.12
base <- 0.5

pop_inflex <- 2.00
pop_sd <- 1.00
pop_width <- 3.00
width_sd <- 1.03

type_inflex <- 2.50
type_width <- 3.5
type_animal_sd <- 0.55
lapse_animal_sd <- 0.50
extra_error_sd <- 0.10

xx <- seq(from = 0, to = n_levels - 1, length.out = 1e3)
plot(x = xx,
     y = base + (1 - lapse - base) *
       plogis(q = width_coef * (xx - pop_inflex) / pop_width),
     xlim = c(0, n_levels - 1),
     ylim = c(0, 1),
     type = 'l',
     col = 'blue',
     xlab = 'Stimulus',
     ylab = 'Proportion correct')
abline(h = c(0, 1, base, 1 - lapse), lty = c(1, 1, 3, 3))

```



```

animal <- rep(x = 1:n_animals, times = n_levels)
animal <- sort(animal)
animal <- rep(x = animal, times = n_types)
stimulus <- rep(x = 1:n_levels - 1, times = n_animals)
stimulus <- rep(x = stimulus, times = n_types)
stype <- rep(x = 1:n_types, times = n_animals * n_levels)
stype <- sort(stype)

animal <- rep(x = animal, times = replicates)
stimulus <- rep(x = stimulus, times = replicates)
stype <- rep(x = stype, times = replicates)

base_bias_animal <- rnorm(n = n_animals, mean = pop_inflex, sd = pop_sd)
width_animal <- rnorm(n = n_animals, mean = pop_width, sd = width_sd)
type_animal <- rnorm(n = n_animals, mean = 1.0, sd = type_animal_sd)
lapse_animal <- plogis(rnorm(n = n_animals,
                             mean = qlogis(lapse),
                             sd = lapse_animal_sd))

response_y <- rep(x = NA,
                  length = n_animals * n_levels * n_types * replicates)
for (ii in seq_along(response_y)) {
  response_y[ii] <- base + (1 - lapse_animal[animal[ii]] - base) *
    plogis(q = width_coef *
            (stimulus[ii] - (base_bias_animal[animal[ii]] +
                              (stype[ii] - 1) * type_inflex)) /
            (width_animal[animal[ii]] + (stype[ii] - 1) *
              type_width * type_animal[animal[ii]])) +
    rnorm(n = 1, mean = 0, sd = extra_error_sd)
}

correct_incorrect <- rbinom(n = length(response_y),
                           size = 1,
                           prob = response_y)

sim_data <- data.frame(
  correct_incorrect = correct_incorrect,
  animal = LETTERS[animal],
  stimulus = stimulus,
  type = factor(stype, labels = c('alpha', 'beta'))
)

summary(sim_data)

```

```

## correct_incorrect  animal      stimulus  type
## Min.   :0.0000    Length:2800    Min.   :0   alpha:1400
## 1st Qu.:0.0000    Class :character  1st Qu.:1   beta :1400
## Median :1.0000    Mode  :character  Median :3
## Mean   :0.6704                    Mean   :3
## 3rd Qu.:1.0000                    3rd Qu.:5
## Max.   :1.0000                    Max.   :6

```

Save and load data

```
root_dir <- file.path(dirname(knitr::current_input(dir = TRUE)), 'Simulated Data')
file_name <- 'simulated_nonlinear_data.csv'
write.csv(file = file.path(root_dir, file_name), x = sim_data, row.names = FALSE)

dta <- read.csv(file = file.path(root_dir, file_name), header = TRUE)
dta <- within(dta, {
  animal <- factor(animal)
  type <- factor(type)
})

summary(dta)
```

```
## correct_incorrect    animal      stimulus    type
## Min.   :0.0000    A      : 280    Min.   :0    alpha:1400
## 1st Qu.:0.0000    B      : 280    1st Qu.:1    beta :1400
## Median :1.0000    C      : 280    Median :3
## Mean   :0.6704    D      : 280    Mean   :3
## 3rd Qu.:1.0000    E      : 280    3rd Qu.:5
## Max.   :1.0000    F      : 280    Max.   :6
##                               (Other):1120
```

Plot the data

We inspect the aggregated proportion correct by stimulus and type.

```
clz <- sapply(length(unique(dta$animal)),
              colorRampPalette(c('red', 'blue', 'cyan4', 'magenta4', 'orange'))))
clz <- sample(clz)

agg <- aggregate(correct_incorrect ~ stimulus * type * animal,
                 data = dta,
                 FUN = mean)

head(agg)
```

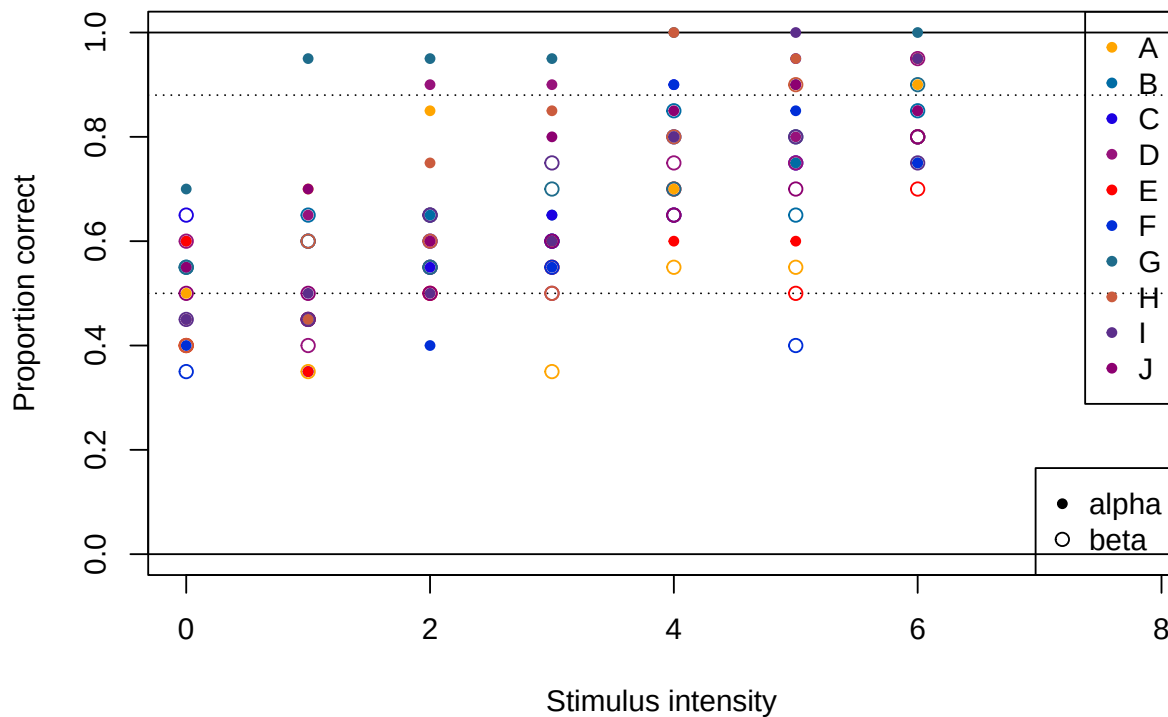
```
## stimulus type animal correct_incorrect
## 1         0 alpha    A              0.50
## 2         1 alpha    A              0.70
## 3         2 alpha    A              0.85
## 4         3 alpha    A              0.60
## 5         4 alpha    A              0.70
## 6         5 alpha    A              0.75
```

```
plot(NULL,
     xlab = 'Stimulus intensity',
     ylab = 'Proportion correct',
     xlim = range(stimulus) * c(0.7, 1.3),
     ylim = c(0, 1))
abline(h = c(0, 1, base, 1 - lapse), lty = c(1, 1, 3, 3))
```

```

for (aa in unique(agg$animal)) {
  with(subset(agg, animal == aa), {
    points(
      x = stimulus,
      y = correct_incorrect,
      col = clz[which(LETTERS %in% aa)],
      pch = c(20, 21)[1 + type %in% levels(type)[2]]
    )
  })
}
legend('topright', legend = unique(agg$animal), col = clz, pch = 20)
legend('bottomright', legend = unique(agg$type), pch = c(20, 21))

```



Install and set up Bayesian modelling packages

This section describes package installation steps for `cmdstanr` and `brms`.

```

if (!any(installed.packages() %in% 'remotes')) {
  install.packages('remotes')}
if (!any(installed.packages() %in% 'cmdstanr')) {
  remotes::install_github('stan-dev/cmdstanr')}
require(cmdstanr)
install_cmdstan(cores = parallel::detectCores() - 1, overwrite = FALSE)

```

```
if (!any(installed.packages() %in% 'brms')) install.packages('brms')
require(cmdstanr)
require(brms)
```

Define the nonlinear model

We specify a nonlinear formula with transformed parameters for baseline, lapse, inflexion, and width.

```
library(brms)
inv_logit <- inv_logit_scaled

formula_nl <- bf(
  correct_incorrect ~ inv_logit(LogitBase) +
    (1 - inv_logit(LogitLapse) - inv_logit(LogitBase)) *
    inv_logit(4.39 * (stimulus - Inflex) / exp(LogWidth)),
  LogitBase ~ 1,
  LogitLapse ~ type + (1 + type | animal),
  Inflex ~ type + (1 + type | animal),
  LogWidth ~ type + (1 + type | animal),
  family = bernoulli('identity'),
  nl = TRUE
)

formulaNull_nl <- bf(
  correct_incorrect ~ inv_logit(LogitBase) +
    (1 - inv_logit(LogitLapse) - inv_logit(LogitBase)) *
    inv_logit(LogitMean),
  LogitBase ~ 1,
  LogitLapse ~ 1 + (1 | animal),
  LogitMean ~ 1 + (1 | animal),
  family = bernoulli('identity'),
  nl = TRUE
)

prior_nl <- get_prior(formula = formula_nl, data = dta)
priorNull_nl <- get_prior(formula = formulaNull_nl, data = dta)
```

Set custom priors

We replace default priors with more informative priors for the nonlinear parameters.

```
prior_nl <- within(prior_nl, {
  prior[class %in% 'b' &
    nlpar %in% 'LogitBase' &
    coef %in% 'Intercept'] <- 'normal(0,1)'
  prior[class %in% 'b' &
    nlpar %in% 'LogitLapse' &
    coef %in% 'Intercept'] <- 'normal(-3,3)'
  prior[class %in% 'b' &
    nlpar %in% 'LogitLapse' &
    coef %in% 'typebeta'] <- 'normal(0,3)'
```

```

prior[class %in% 'b' &
      nlpar %in% 'Inflex' &
      coef %in% 'Intercept'] <- 'normal(3,3)'
prior[class %in% 'b' &
      nlpar %in% 'Inflex' &
      coef %in% 'typebeta'] <- 'normal(0,3)'
prior[class %in% 'b' &
      nlpar %in% 'LogWidth' &
      coef %in% 'Intercept'] <- 'normal(1,3)'
prior[class %in% 'b' &
      nlpar %in% 'LogWidth' &
      coef %in% 'typebeta'] <- 'normal(0,3)'
#prior
})

priorNull_nl <- within(priorNull_nl, {
  prior[class %in% 'b' &
        nlpar %in% 'LogitBase' &
        coef %in% 'Intercept'] <-
    prior_nl$prior[prior_nl$class %in% 'b' &
                  prior_nl$nlpar %in% 'LogitBase' &
                  prior_nl$coef %in% 'Intercept']
  prior[class %in% 'b' &
        nlpar %in% 'LogitLapse' &
        coef %in% 'Intercept'] <-
    prior_nl$prior[prior_nl$class %in% 'b' &
                  prior_nl$nlpar %in% 'LogitLapse' &
                  prior_nl$coef %in% 'Intercept']
  prior[class %in% 'b' &
        nlpar %in% 'LogitLapse' &
        coef %in% 'typebeta'] <-
    prior_nl$prior[prior_nl$class %in% 'b' &
                  prior_nl$nlpar %in% 'LogitLapse' &
                  prior_nl$coef %in% 'typebeta']
  prior[class %in% 'b' &
        nlpar %in% 'LogitMean' &
        coef %in% 'Intercept'] <- 'normal(0,3)'
#prior
})

print(prior_nl)

```

```

##          prior class      coef group resp dpar      nlpar lb ub tag
##          lkj(1)   cor              animal
##          lkj(1)   cor              animal
##          (flat)    b
##          normal(3,3) b Intercept      Inflex
##          normal(0,3) b typebeta      Inflex
## student_t(3, 0, 2.5) sd              Inflex 0
## student_t(3, 0, 2.5) sd              animal Inflex 0
## student_t(3, 0, 2.5) sd Intercept animal Inflex 0
## student_t(3, 0, 2.5) sd typebeta animal Inflex 0
##          (flat)    b              LogitBase

```

```

##          normal(0,1)      b Intercept          LogitBase
##          (flat)          b          LogitLapse
##          normal(-3,3)     b Intercept          LogitLapse
##          normal(0,3)      b typebeta          LogitLapse
## student_t(3, 0, 2.5)      sd          LogitLapse 0
## student_t(3, 0, 2.5)      sd          animal      LogitLapse 0
## student_t(3, 0, 2.5)      sd Intercept animal      LogitLapse 0
## student_t(3, 0, 2.5)      sd typebeta animal      LogitLapse 0
##          (flat)          b          LogWidth
##          normal(1,3)      b Intercept          LogWidth
##          normal(0,3)      b typebeta          LogWidth
## student_t(3, 0, 2.5)      sd          LogWidth 0
## student_t(3, 0, 2.5)      sd          animal      LogWidth 0
## student_t(3, 0, 2.5)      sd Intercept animal      LogWidth 0
## student_t(3, 0, 2.5)      sd typebeta animal      LogWidth 0
##          source
##          default
## (vectorized)
##          default
##          default
##          default
##          default
## (vectorized)
## (vectorized)
## (vectorized)
##          default
##          default
##          default
##          default
##          default
##          default
## (vectorized)
## (vectorized)
## (vectorized)
##          default
##          default
##          default
##          default
## (vectorized)
## (vectorized)
## (vectorized)

```

```
print(priorNull_nl)
```

```

##          prior class      coef group resp dpar      nlpar lb ub tag
##          (flat)      b          LogitBase
##          normal(0,1)  b Intercept          LogitBase
##          (flat)      b          LogitLapse
##          normal(-3,3) b Intercept          LogitLapse
## student_t(3, 0, 2.5) sd          LogitLapse 0
## student_t(3, 0, 2.5) sd          animal      LogitLapse 0
## student_t(3, 0, 2.5) sd Intercept animal      LogitLapse 0
##          (flat)      b          LogitMean
##          normal(0,3)  b Intercept          LogitMean

```



```
## student_t(3, 0, 2.5) sd LogitMean 0
## student_t(3, 0, 2.5) sd animal LogitMean 0
## student_t(3, 0, 2.5) sd Intercept animal LogitMean 0
## source
## default
## default
## default
## default
## default
## (vectorized)
## (vectorized)
## default
## default
## default
## (vectorized)
## (vectorized)
```

Fit the model

The Bayesian model is fit in stages: a prior-only dummy run, a short run, and a full run.

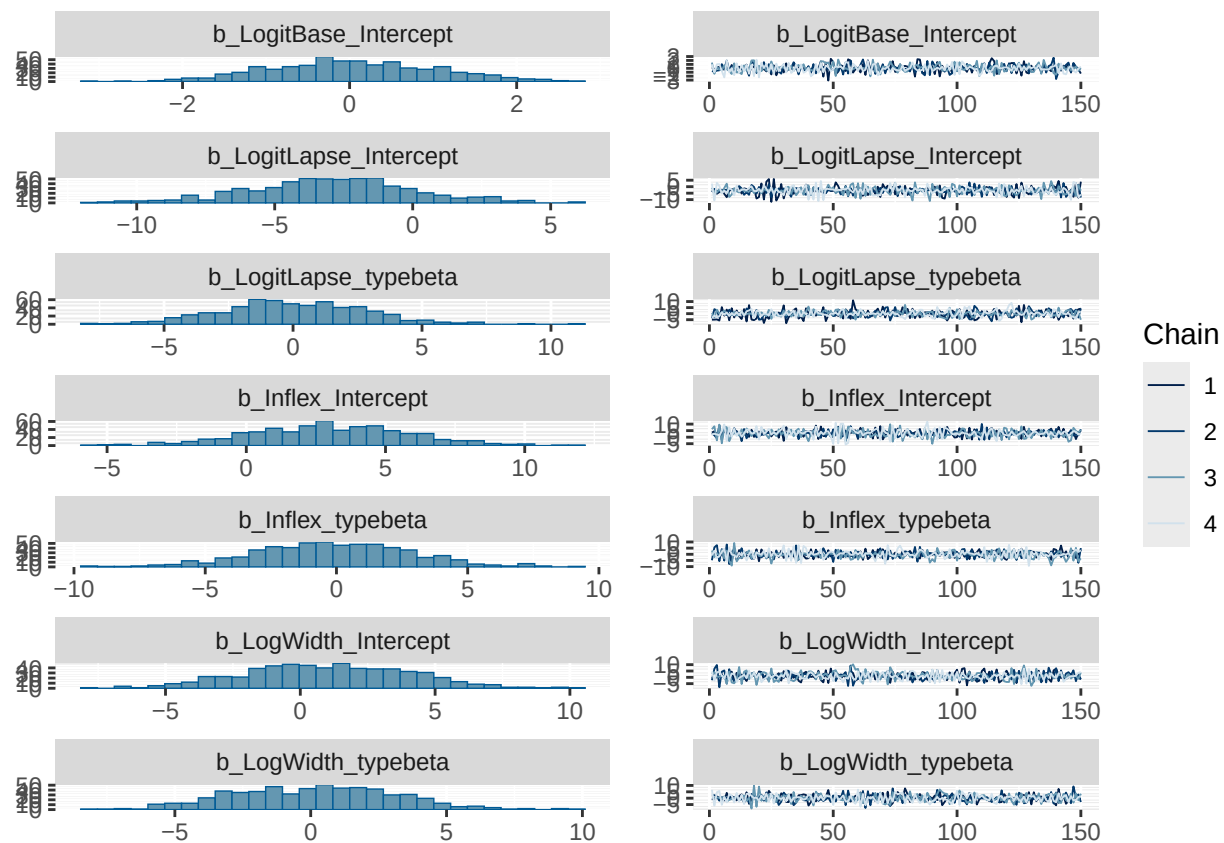
```
# Dummy prior-only fit
dummy_fit <- brm(formula = formula_nl,
  data = dta,
  prior = prior_nl,
  sample_prior = 'only',
  iter = 300, chains = 4,
  cores = 4,
  refresh = 0,
  backend = 'cmdstanr')
```

```
## Start sampling
```

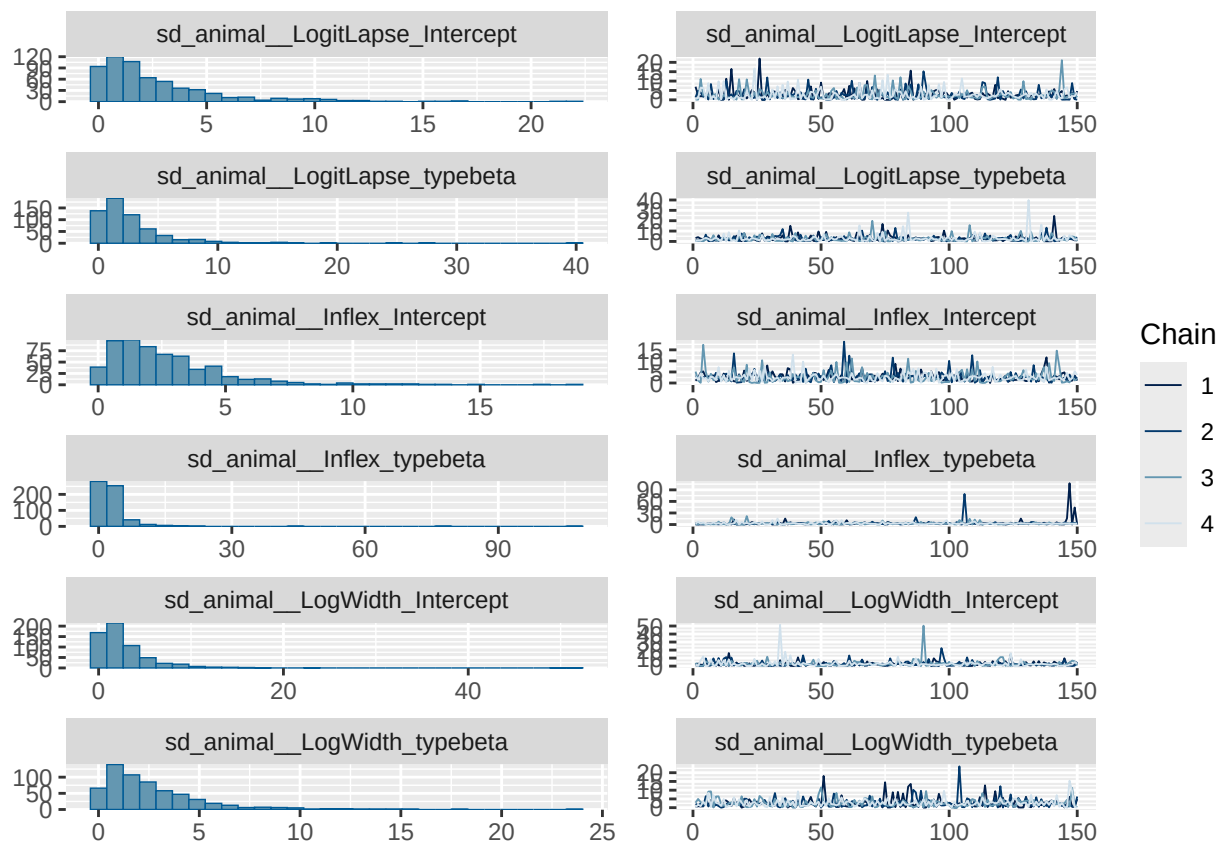
```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 0.1 seconds.
## Chain 2 finished in 0.1 seconds.
## Chain 3 finished in 0.1 seconds.
## Chain 4 finished in 0.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.3 seconds.
```

```
## Loading required namespace: rstan
```

```
plot(dummy_fit, nvariables = 10, variable = '^b_', regex = TRUE)
```



```
plot(dummy_fit, nvariables = 10, variable = '^sd', regex = TRUE)
```



```
# Short fit for convergence check
short_fit <- brm(formula = formula_nl,
  data = dta,
  prior = prior_nl,
  iter = 300,
  chains = 4,
  cores = 4,
  refresh = 0,
  backend = 'cmdstanr')
```

```
## Start sampling
```

```
## Running MCMC with 4 parallel chains...
```

```
##
```

```
## Chain 1 finished in 29.4 seconds.
```

```
## Chain 3 finished in 30.5 seconds.
```

```
## Chain 2 finished in 36.8 seconds.
```

```
## Chain 4 finished in 90.1 seconds.
```

```
##
```

```
## All 4 chains finished successfully.
```

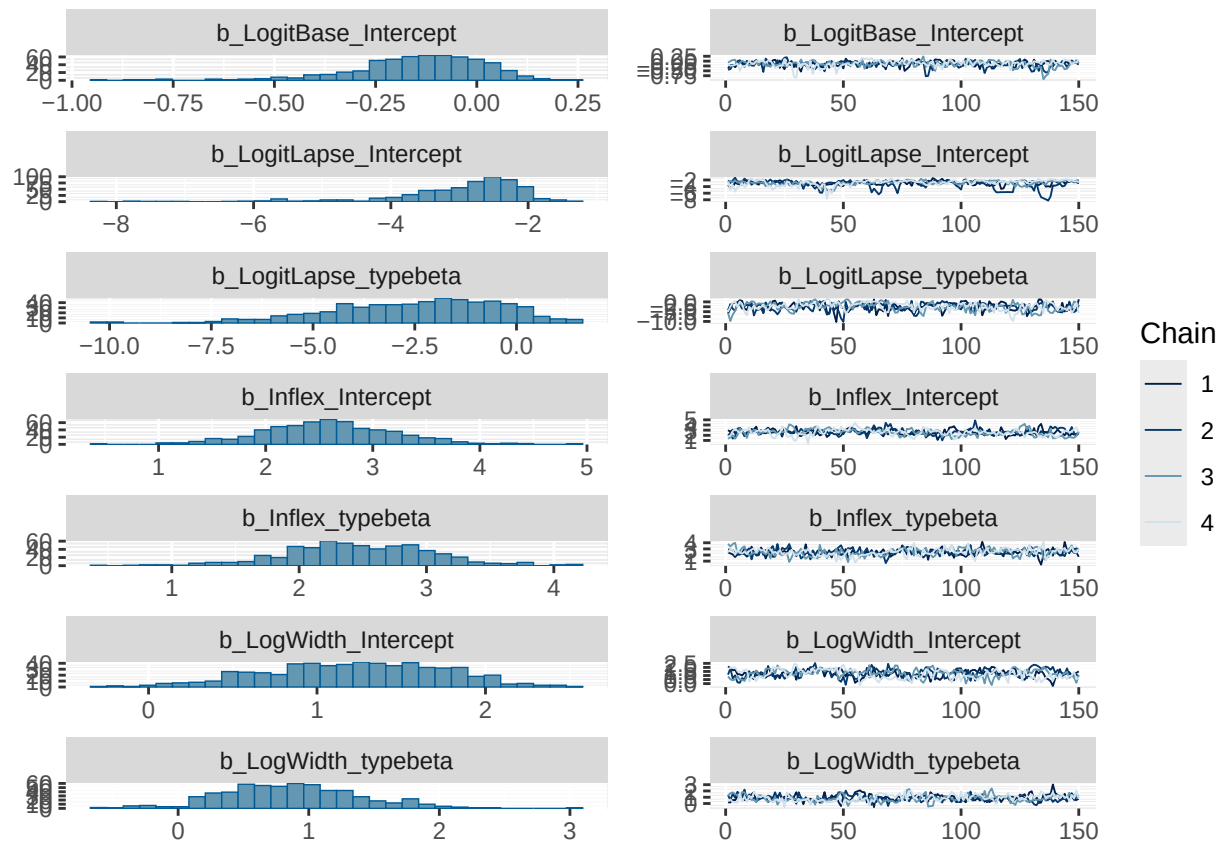
```
## Mean chain execution time: 46.7 seconds.
```

```
## Total execution time: 90.2 seconds.
```

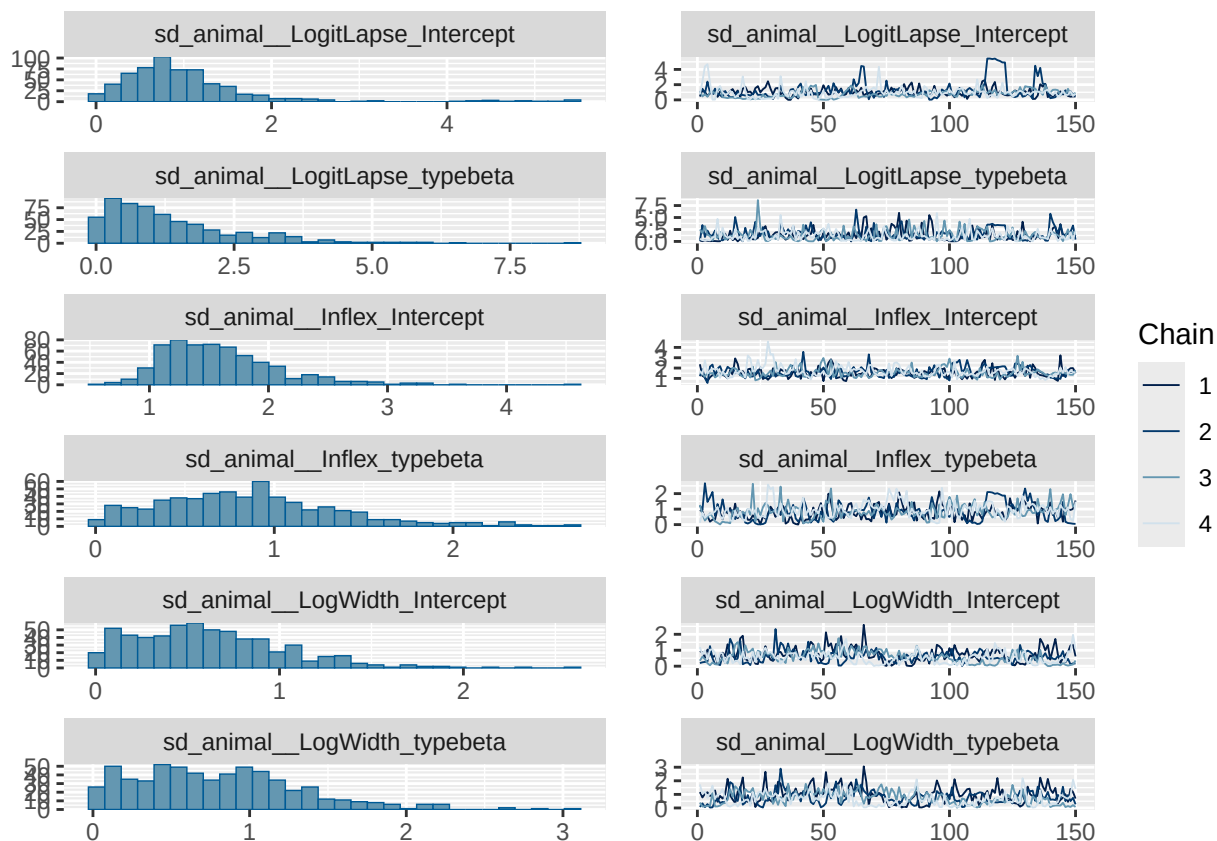
```
## Warning: 2 of 600 (0.0%) transitions ended with a divergence.
```

```
## See https://mc-stan.org/misc/warnings for details.
```

```
plot(short_fit, nvariables = 10, variable = '^b_', regex = TRUE)
```



```
plot(short_fit, nvariables = 10, variable = '^sd', regex = TRUE)
```



```
# Full fit
full_fit <- brm(formula = formula_nl,
  data = dta,
  prior = prior_nl,
  iter = 1000,
  chains = 4,
  cores = 4,
  refresh = 0,
  backend = 'cmdstanr')
```

```
## Start sampling
```

```
## Running MCMC with 4 parallel chains...
```

```
##
```

```
## Chain 2 finished in 130.9 seconds.
```

```
## Chain 3 finished in 131.3 seconds.
```

```
## Chain 4 finished in 154.0 seconds.
```

```
## Chain 1 finished in 301.9 seconds.
```

```
##
```

```
## All 4 chains finished successfully.
```

```
## Mean chain execution time: 179.5 seconds.
```

```
## Total execution time: 302.0 seconds.
```

```
## Warning: 1 of 2000 (0.0%) transitions ended with a divergence.
```

```
## See https://mc-stan.org/misc/warnings for details.
```

```

null_fit <- brm(formula = formulaNull_nl,
  data = dta,
  prior = priorNull_nl,
  iter = 1000,
  chains = 4,
  cores = 4,
  refresh = 0,
  backend = 'cmdstanr')

```

```
## Start sampling
```

```
## Running MCMC with 4 parallel chains...
```

```
##
```

```
## Chain 1 finished in 53.3 seconds.
```

```
## Chain 3 finished in 75.0 seconds.
```

```
## Chain 4 finished in 78.5 seconds.
```

```
## Chain 2 finished in 79.4 seconds.
```

```
##
```

```
## All 4 chains finished successfully.
```

```
## Mean chain execution time: 71.6 seconds.
```

```
## Total execution time: 79.6 seconds.
```

```
## Warning: 13 of 2000 (1.0%) transitions ended with a divergence.
```

```
## See https://mc-stan.org/misc/warnings for details.
```

Inspect the posterior fit

```
full_sm <- summary(full_fit, robust = TRUE)
```

```
## Warning: There were 1 divergent transitions after warmup. Increasing
```

```
## adapt_delta above 0.8 may help. See
```

```
## http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
```

```
print(full_sm)
```

```
## Family: bernoulli
```

```
## Links: mu = identity
```

```
## Formula: correct_incorrect ~ inv_logit(LogitBase) + (1 - inv_logit(LogitLapse) - inv_logit(LogitBase)
```

```
## LogitBase ~ 1
```

```
## LogitLapse ~ type + (1 + type | animal)
```

```
## Inflex ~ type + (1 + type | animal)
```

```
## LogWidth ~ type + (1 + type | animal)
```

```
## Data: dta (Number of observations: 2800)
```

```
## Draws: 4 chains, each with iter = 1000; warmup = 500; thin = 1;
```

```
## total post-warmup draws = 2000
```

```
##
```

```
## Multilevel Hyperparameters:
```

```
## ~animal (Number of levels: 10)
```

```
##
```

```
Estimate Est.Error 1-95% CI
```

```
## sd(LogitLapse_Intercept)          0.88      0.44      0.16
## sd(LogitLapse_typebeta)          1.00      0.95      0.03
## sd(Inflex_Intercept)             1.48      0.41      0.91
## sd(Inflex_typebeta)              0.74      0.52      0.03
## sd(LogWidth_Intercept)           0.57      0.40      0.04
## sd(LogWidth_typebeta)            0.74      0.56      0.04
## cor(LogitLapse_Intercept,LogitLapse_typebeta) -0.09      0.76     -0.97
## cor(Inflex_Intercept,Inflex_typebeta) -0.69      0.33     -0.99
## cor(LogWidth_Intercept,LogWidth_typebeta) -0.83      0.21     -1.00
##                                u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(LogitLapse_Intercept)          2.59 1.00      799      684
## sd(LogitLapse_typebeta)          4.34 1.00     1261      958
## sd(Inflex_Intercept)             2.61 1.00      930     1402
## sd(Inflex_typebeta)              2.01 1.00      411      491
## sd(LogWidth_Intercept)           1.65 1.01      204      531
## sd(LogWidth_typebeta)            2.02 1.02      215      707
## cor(LogitLapse_Intercept,LogitLapse_typebeta) 0.93 1.00     1382     1102
## cor(Inflex_Intercept,Inflex_typebeta) 0.55 1.00     1099     1203
## cor(LogWidth_Intercept,LogWidth_typebeta) 0.76 1.02      243      756
##
## Regression Coefficients:
##              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
## LogitBase_Intercept    -0.11     0.14   -0.51    0.12 1.01      505
## LogitLapse_Intercept   -2.66     0.65   -5.72   -1.73 1.02      169
## LogitLapse_typebeta    -2.11     1.98   -7.10    0.95 1.00      979
## Inflex_Intercept       2.56     0.56    1.40    3.71 1.01      420
## Inflex_typebeta        2.51     0.55    1.27    3.59 1.00      590
## LogWidth_Intercept     1.20     0.58   -0.09    2.17 1.02      194
## LogWidth_typebeta      0.86     0.55   -0.18    2.04 1.02      247
##              Tail_ESS
## LogitBase_Intercept      759
## LogitLapse_Intercept     249
## LogitLapse_typebeta      646
## Inflex_Intercept        769
## Inflex_typebeta         881
## LogWidth_Intercept      540
## LogWidth_typebeta       578
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
full_fix <- full_sm$fixed
full_fix_rn <- rownames(full_fix)

round(plogis(q = with(full_fix[full_fix_rn %in% 'LogitBase_Intercept', ],
                      c(median = Estimate,
                        percentile_2.5 = `l-95% CI`,
                        percentile_97.5 = `u-95% CI`))),
      digits = 2)
```

```
##          median percentile_2.5 percentile_97.5
##          0.47          0.38          0.53
```

```
round(plogis(q = with(full_fix[full_fix_rn %in% 'LogitLapse_Intercept', ],
                      c(median = Estimate,
                        percentile_2.5 = `l-95% CI`,
                        percentile_97.5 = `u-95% CI`))),
      digits = 2)
```

```
##           median percentile_2.5 percentile_97.5
##           0.07           0.00           0.15
```

```
round(exp(with(full_fix[full_fix_rn %in% 'LogWidth_Intercept', ],
                c(median = Estimate,
                  percentile_2.5 = `l-95% CI`,
                  percentile_97.5 = `u-95% CI`))),
      digits = 2)
```

```
##           median percentile_2.5 percentile_97.5
##           3.32           0.91           8.75
```

```
round(with(full_fix[full_fix_rn %in% 'Inflex_Intercept', ],
            c(median = Estimate,
              percentile_2.5 = `l-95% CI`,
              percentile_97.5 = `u-95% CI`))),
      digits = 2)
```

```
##           median percentile_2.5 percentile_97.5
##           2.56           1.40           3.71
```

```
round(with(full_fix[full_fix_rn %in% 'Inflex_typebeta', ],
            c(median = Estimate,
              percentile_2.5 = `l-95% CI`,
              percentile_97.5 = `u-95% CI`))),
      digits = 2)
```

```
##           median percentile_2.5 percentile_97.5
##           2.51           1.27           3.59
```

Model comparison and predictions

```
loo_full <- loo(full_fit)
loo_null <- loo(null_fit)
loo_compare(loo_full, loo_null)
```

```
##           elpd_diff se_diff
## full_fit      0.0         0.0
## null_fit    -127.0        14.8
```



```

full_cond <- brms::conditional_effects(full_fit,
                                     method = 'posterior_epred',
                                     cores = parallel::detectCores() - 1,
                                     effects = c('stimulus:type'))

pred_data <- full_cond$`stimulus:type`

plot(NULL, xlab = 'Stimulus intensity',
      ylab = 'Proportion correct',
      xlim = range(stimulus) * c(0.7, 1.3),
      ylim = c(0, 1))
abline(h = c(0, 1, base, 1 - lapse),
      lty = c(1, 1, 3, 3))
for (aa in unique(agg$animal)) {
  with(subset(agg, animal == aa), {
    points(x = stimulus, y = correct_incorrect,
           col = c('blue', 'orange3')[1 + type %in% levels(type)[2]],
           pch = c(20, 21)[1 + type %in% levels(type)[2]])
  })
}
with(subset(pred_data, type == 'beta'), {
  polygon(x = c(sort(stimulus), rev(sort(stimulus))),
         y = c(lower__[order(stimulus)], rev(upper__[order(stimulus)])),
         col = adjustcolor('orange', alpha.f = 50 / 256), border = NA)
})
with(subset(pred_data, type == 'alpha'), {
  polygon(x = c(sort(stimulus), rev(sort(stimulus))),
         y = c(lower__[order(stimulus)], rev(upper__[order(stimulus)])),
         col = adjustcolor('darkblue', alpha.f = 50 / 256), border = NA)
})
with(subset(pred_data, type == 'alpha'),
     lines(x = sort(stimulus),
           y = estimate__[order(stimulus)],
           col = 'darkblue', lwd = 5))
with(subset(pred_data, type == 'beta'),
     lines(x = sort(stimulus),
           y = estimate__[order(stimulus)],
           col = 'orange', lwd = 5))
legend('bottomright',
      legend = unique(agg$type),
      col = c('blue', 'orange3'),
      pch = c(20, 21))

```

